PHP + MySQL Blog App

Simple CRUD Project — Create, View, Update, Delete

1. Project Overview

This is a beginner-friendly Blog application built with PHP and MySQL. It demonstrates the four basic CRUD operations — Create, Read, Update, and Delete — using a single database table called **posts**. The app uses **prepared statements** (via mysqli) to prevent SQL injection, and **htmlspecialchars()** to prevent XSS when displaying data.

2. Features

- Create a new blog post
- View all posts (list page)
- View a single post in detail
- Update / Edit an existing post
- Delete a post
- MySQL database with prepared statements (secure)
- Automatic Created Date & Updated Date columns

3. Folder Structure

```
blog-app/
|
|-- config/
|      `-- db.php          (database connection)
|
|-- index.php              (view all posts)
|-- create.php              (create new post)
|-- view.php                (view single post)
|-- edit.php                (update post)
|-- delete.php              (delete post)
|
`-- assets/
       `-- style.css        (optional styling)
```

4. Database Setup (SQL)

Run this SQL once in phpMyAdmin / MySQL CLI to create the database and table.

```
CREATE DATABASE blog_app;
USE blog_app;

CREATE TABLE posts (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    title      VARCHAR(255) NOT NULL,
    content    TEXT NOT NULL,
    author     VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
               ON UPDATE CURRENT_TIMESTAMP
);
```

5. Database Connection

File: config/db.php

This file connects PHP to MySQL using mysqli. Every other page will include this file first.

```
<?php
$host = "localhost";
$user = "root";
$pass = "";
$db   = "blog_app";

$conn = new mysqli($host, $user, $pass, $db);

if ($conn->connect_error) {
    die("Connection Failed: " . $conn->connect_error);
}
?>
```

6. Create Post

File: create.php

Shows a form to add a new post. On submit, it inserts the data into the **posts** table using a prepared statement, then redirects to index.php.

```php
<?php
include 'config/db.php';

if (isset($_POST['save'])) {
    $title   = $_POST['title'];
    $content = $_POST['content'];
    $author  = $_POST['author'];

    $stmt = $conn->prepare(
        "INSERT INTO posts (title, content, author) VALUES (?, ?, ?)"
    );
    $stmt->bind_param("sss", $title, $content, $author);
    $stmt->execute();

    header("Location: index.php");
    exit;
}
?>
<h2>Create New Post</h2>
<form method="POST">
    <input type="text" name="title" placeholder="Title" required>
    <br><br>
    <input type="text" name="author" placeholder="Author">
    <br><br>
    <textarea name="content" rows="8" required></textarea>
    <br><br>
    <button name="save">Save Post</button>
</form>
<a href="index.php">Back</a>
```

7. View All Posts

File: index.php

Fetches every row from **posts** (newest first) and lists them with View, Edit and Delete links.

```php
<?php
include 'config/db.php';

$result = $conn->query("SELECT * FROM posts ORDER BY id DESC");
?>
<h2>Blog Posts</h2>
<a href="create.php">Create New Post</a>
<hr>

<?php while ($row = $result->fetch_assoc()): ?>
    <h3><?= htmlspecialchars($row['title']) ?></h3>
    <p>By <?= htmlspecialchars($row['author']) ?></p>

    <a href="view.php?id=<?= $row['id'] ?>">View</a>
    <a href="edit.php?id=<?= $row['id'] ?>">Edit</a>
    <a href="delete.php?id=<?= $row['id'] ?>">Delete</a>
    <hr>
<?php endwhile; ?>
```

8. View Single Post

File: view.php

Reads the **id** from the URL (?id=5), fetches that one post using a prepared statement, and displays its full content.

```
<?php
include 'config/db.php';

$id = (int) $_GET['id'];

$stmt = $conn->prepare("SELECT * FROM posts WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$post = $stmt->get_result()->fetch_assoc();
?>

<h2><?= htmlspecialchars($post['title']) ?></h2>
<p>By <?= htmlspecialchars($post['author']) ?></p>
<p><?= nl2br(htmlspecialchars($post['content'])) ?></p>

<a href="index.php">Back</a>
```

9. Edit / Update Post

File: edit.php

First loads the existing post into the form fields. When the **update** button is pressed, it runs an UPDATE query for that specific id and redirects back to index.php.

```php
<?php
include 'config/db.php';

$id = (int) $_GET['id'];

if (isset($_POST['update'])) {
    $title   = $_POST['title'];
    $content = $_POST['content'];
    $author  = $_POST['author'];

    $stmt = $conn->prepare(
        "UPDATE posts SET title=?, content=?, author=? WHERE id=?"
    );
    $stmt->bind_param("sssi", $title, $content, $author, $id);
    $stmt->execute();

    header("Location: index.php");
    exit;
}

// Load existing post first
$stmt = $conn->prepare("SELECT * FROM posts WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$post = $stmt->get_result()->fetch_assoc();
?>

<h2>Edit Post</h2>
<form method="POST">
    <input type="text" name="title"
           value="<?= htmlspecialchars($post['title']) ?>">
    <br><br>
    <input type="text" name="author"
           value="<?= htmlspecialchars($post['author']) ?>">
    <br><br>
    <textarea name="content" rows="8">
<?= htmlspecialchars($post['content']) ?>
    </textarea>
    <br><br>
    <button name="update">Update Post</button>
</form>
<a href="index.php">Back</a>
```

10. Delete Post

File: delete.php

Deletes the post matching the given id, then redirects back to the post list.

```php
<?php
include 'config/db.php';

$id = (int) $_GET['id'];

$stmt = $conn->prepare("DELETE FROM posts WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();

header("Location: index.php");
exit;
?>
```

11. How It All Works Together

- **index.php** is the homepage — it lists every post from the database.
- Clicking **Create New Post** opens **create.php**, which saves a new row.
- Clicking **View** on any post opens **view.php?id=X** to show full details.
- Clicking **Edit** opens **edit.php?id=X**, pre-filled with that post's data.
- Clicking **Delete** opens **delete.php?id=X**, which removes the row instantly.
- Every page starts with **include 'config/db.php'** to connect to MySQL.

12. Advanced Features (Optional / Later)

Once the basic CRUD app works, these can be added to make it a full portfolio-level project:

- User Login / Register system
- Blog Categories and Tags
- Featured Image Upload
- Rich Text Editor (e.g. CKEditor)
- Comments System
- Search Posts
- Pagination
- SEO-friendly URL slugs
- Admin Dashboard
- Dark Mode
- REST API version

Tip: Type each file's code in this order — db.php, create.php, index.php, view.php, edit.php, delete.php — testing each one in the browser before moving to the next.